

C_Topic_analyse

March 12, 2025

```
[1]: import pandas as pd
from ast import literal_eval
from tqdm import tqdm; tqdm.pandas()
import numpy as np
import torch
from transformers import set_seed
from collections import Counter
import matplotlib.pyplot as plt
from bertopic import BERTopic
from umap import UMAP
from hdbscan import HDBSCAN
from sentence_transformers import SentenceTransformer
from bertopic.vectorizers import ClassTfidfTransformer
```

```
/home/rdubel/.local/lib/python3.8/site-packages/tqdm/auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

```
[12]: df = pd.read_csv("df3.csv")
```

```
[14]: # Deze code groepeer documenten op basis van hun inhoud en vindt de
      ↳ onderwerpen in de teksten.
      # De documenten worden in groepen (clustertjes) ingedeeld op basis van
      ↳ gelijkenissen.
      # Het model kijkt naar veelvoorkomende woorden en vermindert de invloed daarvan.
      ↳
      # Het resultaat is een set van groepen met documenten die over dezelfde
      ↳ onderwerpen gaan.
      # Ongebruikelijke documenten die niet goed in een groep passen, worden
      ↳ verwijderd.

docs = df.text.to_list()

umap_model = UMAP(n_neighbors=10, n_components=5, min_dist=0.1,
↳ metric='euclidean', random_state=1)

embedding_model = SentenceTransformer("BAAI/bge-base-en-v1.5")
```

```

hdbscan_model = HDBSCAN(min_cluster_size=20, metric='euclidean',
    ↳cluster_selection_method='eom', prediction_data=True, min_samples=5)

ctfidf_model = ClassTfidfTransformer(reduce_frequent_words=True)

np.random.seed(1)
torch.manual_seed(1)
set_seed(1)

topic_model = BERTopic(umap_model=umap_model,
    language="multilingual",
    embedding_model=embedding_model,
    hdbscan_model=hdbscan_model,
    ctfidf_model=ctfidf_model,
    min_topic_size=20)

topics, probs = topic_model.fit_transform(docs)
new_topics = topic_model.reduce_outliers(docs, topics)

```

```

[15]: # Deze code voegt de toegewezen onderwerpnaam toe aan de dataset voor elk
    ↳document.
# Het model, dat eerder getraind is, wordt gebruikt om te bepalen bij welk
    ↳onderwerp elk document hoort.
# De naam van het onderwerp wordt voor elk document opgehaald.

df["topic_bert"] = topic_model.get_document_info(docs)["Name"].values

```

```

[22]: # Deze code geeft een naam die past bij een groep.

```

```

def interpret_clusters(x):
    if x[:2] == "-1":
        return "Overig"
    if x[:2] in ["0_", "5_", "8_"]:
        return "Demonstraties"
    if x[:2] == "1_":
        return "Jongeren"
    if x[:2] in ["2_", "18"]:
        return "Drugs"
    if x[:2] == "6_":
        return "Voetbal"
    if x[:2] == "7_":
        return "Discriminatie"
    if x[:2] == "9_":
        return "Verkeer"
    if x[:2] == "10":
        return "Vuurwerk"

```

```

if x[:2] == "11":
    return "Explosies"
if x[:2] in ["12", "14"]:
    return "Seksueel misbruik"
if x[:2] == "13":
    return "Communicatie"
if x[:2] == "15":
    return "Vuurwapen geweld"
else:
    return x[:2]

```

```

[23]: df["merged_topics"] = df.topic_bert.progress_apply(lambda x:
↳ interpret_clusters(x))

```

```

100%|          | 1304/1304 [00:00<00:00, 181936.41it/s]

```

```

[25]: len(df[df.merged_topics != "other"])

```

```

[25]: 1304

```

```

[29]: df.to_csv("df4.csv")

```